

Arquitetura do Aplicativo do Prestador no BarberTime: Clean Architecture e Notificação Assíncrona por Polling

Caio Victor Kodato Teixeira

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Curso de Engenharia de Software — Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas (LDAMD)

Belo Horizonte — MG — Brasil

Projeto Integrador — Documento de Arquitetura (Sprint 4)

Tecnologias: Flutter/Dart, pacote provider, package:http.

Resumo. Este documento detalha a arquitetura do aplicativo móvel do prestador (barbeiro) do BarberTime, implementado em Flutter segundo uma Clean Architecture simplificada. Descrevem-se as camadas, o fluxo unidirecional de dados, o mecanismo de atualização assíncrona de estado por *polling* com detecção incremental de novas solicitações e as ações de aceite e recusa. Apresentam-se ainda o mapeamento das telas aos requisitos da sprint e as principais decisões de design.

1. Camadas e dependências

O aplicativo do prestador reutiliza a mesma estrutura do aplicativo cliente, organizada em cinco camadas. A regra da dependência aponta para o interior: a apresentação conhece os dados e os modelos; a camada de dados conhece modelos e utilitários; modelos e *core* não conhecem as camadas externas (Martin, 2019). A interface nunca executa HTTP diretamente — sempre por meio de AppState → Repository → ApiClient. O Listing 1 mostra a estrutura de diretórios e a Tabela 1 resume a responsabilidade de cada camada.

```
lib/  
├── config/           # app_config.dart (URL da API, polling = 8 s)  
├── core/  
│   ├── errors/      # api_exception.dart  
│   ├── theme/       # app_theme.dart (identidade BarberTime)  
│   └── time/        # app_time.dart (fuso America/Sao_Paulo)  
├── models/          # user, auth_result, barber_appointment  
├── data/  
│   ├── api/         # api_client.dart (http + JWT + timeout)  
│   └── repositories/ # auth_repository, barber_appointment_repository  
└── presentation/  
    ├── providers/    # app_state.dart (sessão, polling, alertas)  
    ├── screens/      # app_root, login, home_shell, lista, detalhe  
    └── widgets/      # brand_logo, status_badge, request_card
```

Listing 1. Estrutura de pastas do app prestador (mobile_provider/lib).

Tabela 1. Camadas do aplicativo e responsabilidades.

Camada	Responsabilidade
config	Parâmetros (URL base da API, intervalo de polling, chaves de preferências).
core	Tema visual, fuso horário e exceções de API.
models	Entidades imutáveis: User, AuthResult, BarberAppointment.
data	Cliente HTTP e repositórios que adaptam as rotas REST a modelos.
presentation	Estado (provider), telas e widgets.

2. Estado e atualização assíncrona

O AppState (um ChangeNotifier) mantém a sessão (JWT e usuário, persistidos em SharedPreferences) e a lista de agendamentos. A atualização de estado vinda do servidor é assíncrona por *polling*: um temporizador periódico de oito segundos dispara a recarga da fila. A cada ciclo, o estado compara os identificadores pendentes recém-obtidos com os já conhecidos; identificadores inéditos correspondem a solicitações que chegaram via evento appointment.requested publicado no MOM e geram alertas consumidos pela tela inicial, que exibe um *badge* numérico e um aviso (SnackBar). O Listing 2 resume o fluxo.

```

1. Cliente: POST /appointments -> status pending
2. Backend publica appointment.requested no RabbitMQ
3. App prestador (Timer 8 s): GET /barber/appointments?status=all
4. _detectNewRequests() compara IDs pendentes com os conhecidos
5. IDs novos -> badge++ e SnackBar "Nova solicitação"

```

Listing 2. Ciclo de notificação assíncrona ao prestador.

Optou-se por *polling* em vez de *WebSocket* ou *push* pela simplicidade operacional e robustez em redes móveis, sem manter conexão persistente. O intervalo do prestador (oito segundos) é menor que o do cliente (dez segundos), pois a latência de atendimento é mais sensível para quem recebe o pedido. O RabbitMQ permanece como fonte de verdade para o disparo dos eventos no backend; o aplicativo observa o efeito desses eventos no estado via REST.

3. Ações do prestador

As ações de escrita são expostas pelo AppState e delegadas ao repositório. Após cada mutação, a fila é recarregada para manter coerência com o servidor. A Tabela 2 relaciona as ações aos endpoints e efeitos.

Tabela 2. Ações do prestador, endpoints e efeitos.

Ação	Método em AppState	Endpoint	Efeito
Aceitar	confirmAppointment(id)	PATCH /barber/appointments/:id/confirm	pending → confirmed; publica appointment.confirmed
Recusar	rejectAppointment(id)	PATCH /barber/appointments/:id/reject	pending → cancelled; publica appointment.rejected

4. Telas e mapeamento de requisitos

O aplicativo possui telas de Solicitações, Detalhe da solicitação, Em andamento e Histórico. A Tabela 3 mapeia cada requisito da Sprint 4 ao recurso correspondente.

Tabela 3. Telas do app prestador × requisitos da Sprint 4.

Requisito	Tela / recurso
Lista de solicitações pendentes	Aba <i>Solicitações</i>
Detalhe com aceitar/recusar	Tela <i>Detalhe da solicitação</i>
Acompanhamento das solicitações em andamento	Abas <i>Em andamento e Histórico</i>
Notificação assíncrona ao prestador	<i>Polling</i> + badge + <i>SnackBar</i>

5. Decisões de design

- **Reuso de padrão entre apps:** cliente e prestador compartilham camadas, *ApiClient*, tema e utilitário de tempo, reduzindo custo cognitivo e evidenciando a Clean Architecture.
- **Polling com detecção incremental:** a notificação assíncrona é obtida comparando conjuntos de identificadores, sem dependência de serviços externos de *push*.
- **Validação de papel no login:** o aplicativo recusa contas que não sejam barber, evitando uso indevido do painel.
- **Recusar como evento próprio:** a recusa reaproveita o status *cancelled* do schema, mas publica um evento distinto (*appointment.rejected*) para notificar o cliente com a mensagem adequada.

Referências

MARTIN, R. C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019.

BAILEY, T. *Flutter for beginners*. 3rd ed. Birmingham: Packt, 2023.